

# Introduction to Algorithm Engineering

Spring 2026

Lecture 06

# Strongly Connected Components

---

- Given a directed graph  $G = (V, E)$ , a strongly connected component of  $G$  is a maximal subset  $S$  of  $V$  such that :
  - Every pair of vertices  $u, v$  in  $S$  have a directed path from  $u$  to  $v$ , and from  $v$  to  $u$ .
- The partitioning of  $V$  into such sets is called as the SCC decomposition of  $G$ .
- One way to obtain such a decomposition is to use the algorithm of Kosaraju that works as follows:
  - Find a DFS of  $G$ .
  - Find a DFS of  $G^T$  with source vertices picked according to the finish time of vertices in the DFS of  $G$ .

# Strongly Connected Components

---

- Finding the SCCs in a parallel setting is much more complicated.
- DFS approach does not work.
- Here are some parallel approaches over the years.

# Notation

---

- Let  $\text{SCC}(v)$  denote the strongly connected component that  $v$  belongs to.
- The descendants of  $v$ ,  $\text{Desc}(G, v)$ , is the subset of vertices in  $G$  which are reachable from  $v$ .
- the predecessors of  $v$ ,  $\text{Pred}(G, v)$ , is the subset of vertices from which  $v$  is reachable.
- The set of vertices that is neither reachable from  $v$  nor reach  $v$  is called the remainder, denoted by  $\text{Rem}(G, v)$ .
  - $\text{Rem}(G, v) = V \setminus \{\text{Desc}(G, v) \cup \text{Pred}(G, v)\}$ .

# Some Claims

---

- Lemma: Let  $G = (V, E)$  be a directed graph, with  $v$  a vertex in  $G$ . Then  $\text{SCC}(G, v) = \text{Desc}(G, v) \cap \text{Pred}(G, v)$ .
- Proof: By definition.

# Some Claims

---

- Let  $G$  be a graph with vertex  $v$ . **Any** strongly connected component of  $G$  is a subset of  $\text{Desc}(G, v)$ , a subset of  $\text{Pred}(G, v)$  or a subset of  $\text{Rem}(G, v)$ .
- Proof: Consider any strongly connected component of  $G$  and two vertices  $u$  and  $w$  in that SCC,  $S$ .
- Note that  $u, w$  are in  $S$  implies that there is a directed path from  $u$  to  $w$  and vice-versa in  $S$ .
- Now, we claim that  $u$  is in  $\text{Desc}(G, v)$  if and only  $w$  is in  $\text{Desc}(G, v)$ .
- Same such claim allows us to write that  $u$  is in  $\text{Pred}(G, v)$  if and only  $w$  is in  $\text{Pred}(G, v)$ , and
- $u$  is in  $\text{Rem}(G, v)$  if and only  $w$  is in  $\text{Rem}(G, v)$ .

# The Algorithm

---

- Select a random vertex  $v$ . Call  $v$  a pivot vertex.
- Find its descendant and predecessor sets.
- The intersection of these sets is  $\text{SCC}(G, v)$ .
- The remaining vertices are divided into three sets  $\text{Desc}(G, v)$ ,  $\text{Pred}(G, v)$ , and  $\text{Rem}(G, v)$ .
- From the above claim, the SCCs contained in these three sets are mutually disjoint.
- So, recursively solve the problem on these three subsets.

# Pseudocode

*DCSC*(*G*)

**If** *G* is empty **then Return.**

**Select** *v* uniformly at random from *V*.

$SCC \leftarrow Pred(G, v) \cap Desc(G, v)$

**Output** *SCC*.

*DCSC*( $\langle Pred(G, v) \setminus SCC \rangle$ )

*DCSC*( $\langle Desc(G, v) \setminus SCC \rangle$ )

*DCSC*( $\langle Rem(G, v) \rangle$ )

# Run time

---

- In the worst case, the run time is  $O(n^2)$ .
- Akin to quick sort.
- The expected behavior depends on the size of the three subproblems.
- In each recursive level, the time spent in identifying the SCC of the pivot vertex is in  $O(n)$ .
- The expected run time is in  $O(n \log n)$ .

# Further Enhancement

---

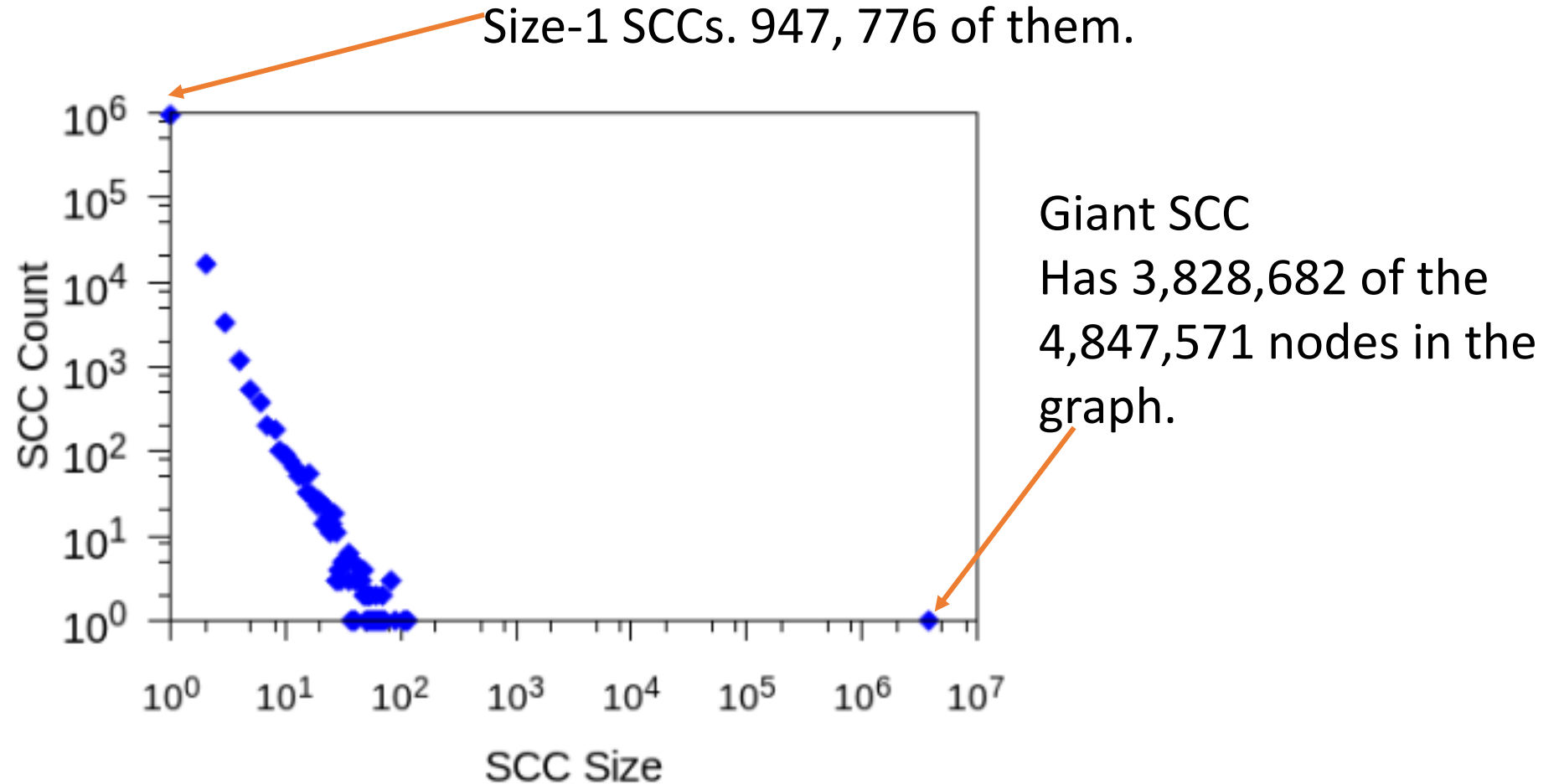
- Motivated by the fact that a trivial SCC, an SCC that contains a single vertex, is easy to identify.
- Such a vertex has either zero incoming edges or zero outgoing edges in the partition identified in a given recursive level.
- Need not initiate a recursive call from these trivial SCCs.
  - This is called as the Trim step.

# Properties of Real-World Graphs

---

- While the above approach suggested by Fleischer et al. is from 2000, there is one notable engineering development in 2013.
  - Also optimizations for parallel processing
- The diameter of such graphs is very small even for very large graph instance
  - This is called as the small-world property.
- In real-world graphs there exists one giant SCC whose size is  $O(n)$ 
  - $n$  is the number of nodes in the graph.
- Beyond the giant SCC, the remaining SCCs have a small size.
- The number of small sized SCCs is much more than the large-sized ones.

# SCC Size Vs Count



**Figure 2: Distribution of SCC sizes in the LiveJournal network.**

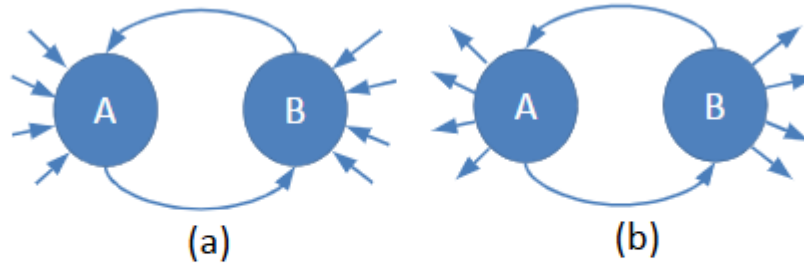
# Parallel Trimming

---

- Attempt to parallelize the prior algorithm.
- While the three recursive calls are independent and can be issued in parallel, the trim step needs some study.
- There are many size-1 SCCs in a real-world graph.
- The parallel trim step greatly increases the degree of parallelism by identifying these SCCs before executing the FW-BW algorithm.
- Although Par-Trim is invoked once at the beginning of Algorithm 3, the actual trimming is iteratively applied inside the Par-Trim kernel.

# Trim2

---



**Figure 4:** Patterns of size-2 SCCs detected by Trim2. There is a tight cycle between A and B but either (a) there are no other outgoing edges from A and B, or (b) there are no other incoming edges to A and B.

- The idea is that a large subset of size-2 SCCs can be detected easily by looking only at the neighbors of a given node.
- Figure 4 illustrates the patterns of size-2 SCCs identified by this algorithm.
- The algorithm first identifies all of the nodes which have a single neighborhood node that is both an incoming neighbor and an outgoing neighbor, i.e. nodes A and B in Figure 4.